

Sequential Types as a Universal Basis for CRDT Design

Anonymous Author(s)

Abstract

Placeholder text

ACM Reference Format:

Anonymous Author(s). 2026. Sequential Types as a Universal Basis for CRDT Design. In . ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Conflict-free Replicated Data Types (CRDTs) have emerged as a fundamental building block for distributed systems, enabling replicas to achieve eventual consistency without coordination. By construction, CRDTs guarantee that concurrent updates commute, allowing replicas to apply operations in any order while converging to the same state. This property makes CRDTs particularly attractive for geo-replicated systems, collaborative applications, and edge computing scenarios where low latency and high availability are paramount.

Despite their theoretical elegance and practical utility, designing CRDTs remains a challenging endeavor. Each new data type requires careful consideration of how to encode operations, resolve conflicts, and ensure convergence. The design process is often ad-hoc and error-prone, requiring deep expertise in concurrency theory and distributed systems. While several CRDTs for common data types (counters, sets, registers) are well-established, extending these principles to arbitrary sequential data types has proven difficult. This raises a fundamental question: *can we systematically derive CRDTs from their sequential counterparts?*

In this paper, we address this question by establishing a formal connection between sequential types and CRDTs through the lens of *liftability*. We show that sequential types possessing certain algebraic properties can be systematically lifted to CRDTs, providing a principled approach to CRDT design. Our main contribution is a characterization theorem demonstrating that a sequential type is liftable to a CRDT if and only if it forms a commutative monoid under its induced binary operator. This result provides both a necessary and

sufficient condition for liftability, offering designers a clear criterion for determining whether a sequential type can be transformed into a CRDT.

The implications of this characterization are far-reaching. First, it provides a framework for understanding existing CRDTs, showing how they arise naturally from the algebraic structure of their sequential counterparts. Second, it offers a systematic methodology for designing new CRDTs: identify the algebraic properties of the sequential type, verify the commutative monoid conditions, and construct the lifted CRDT accordingly. Third, it reveals fundamental limitations, identifying sequential types that cannot be directly lifted and thus require alternative replication strategies.

Contributions. We make the following contributions:

- We formalize the notion of liftability, defining when a sequential type can be systematically transformed into a CRDT while preserving its observable behavior.
- We prove a characterization theorem establishing that liftability is equivalent to the sequential type forming a commutative monoid.
- We demonstrate how this framework unifies and explains existing CRDT designs, providing insights into their algebraic foundations.
- We discuss implications for CRDT design methodology and identify classes of sequential types that admit or preclude lifting.

Organization. The remainder of this paper is organized as follows. Section 2 provides necessary background on sequential types, CRDTs, and algebraic structures. Section 3 presents our formal framework for liftability and proves the characterization theorem. We conclude with a discussion of related work and future directions.

2 Background

3 CRDT Liftability

►(Pierre:) Explain and illustrate the key definitions◄

Definition 3.1 (Sequential Type). A sequential type $T = (S, s_0, \Delta)$ is a triple where S is the set of states of an object of type T , $s_0 \in S$ the initial state and $\Delta : S \times Op \rightarrow S$ is the transition function mapping a state and an operation to a successor state.

We write $s \xrightarrow{op} s'$ when $\Delta(s, op) = s'$

Definition 3.2 (Induced binary operator). Given a sequential type T , we define the binary operator $^1 \oplus : S \times S \rightarrow S$

¹The binary operator $\oplus$ is well-defined only when the transition system is confluent. ►(Pierre:) define this term and discuss what does it implies.◄

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

induced by the transition function Δ such that for any states $s, s', s'' \in S$, $s \oplus s' = s''$ by applying the union of the operation multi-sets that lead to states s and s' respectively.

Formally, if $s_0 \xrightarrow{op_1 \dots op_m} s$ and $s_0 \xrightarrow{op'_1 \dots op'_n} s'$. Then, $s \oplus s' = \Delta^*(s_0, \{op_1 \dots op_m\} \uplus \{op'_1 \dots op'_n\})$

►(Pierre:) Δ^* is not defined.◄

►(Pierre:) In this definition, operation op'_1 has to be applicable to the last state reached by op_m . This requires the transsition function to be total.◄

►(Pierre:) What about when there are several sequences leading to the same state? Do we take a canonical one?◄

►(Ayush:) The original definition of CRDT also defines a query operator in the tuple but never formally specifies its details. Do we include it here?◄

Definition 3.3 (CRDT). A conflict-free replicated datatype, denoted $\mathbb{T} := (\mathbb{S}, \leq, \perp, u, \sqcup)$, is a tuple consisting of a set of states $\mathbb{S}$, partially ordered by a comparison operator $\leq$ and an initial state $\perp \in \mathbb{S}$. Other states are derived from this initial state via a set of update operations u . CRDT state is monotonically non-decreasing across updates i.e. $u(s) = s' \Rightarrow s \leq s'$. A binary merge operator $\sqcup : \mathbb{S} \times \mathbb{S} \rightarrow \mathbb{S}$ computes the lowest upper bound of two states in $\mathbb{S}$.

Definition 3.4 (Liftability). A sequential type T admits a $\blacktriangle$ -lifting if there exists a CRDT type $T^\blacktriangle = (\mathbb{S}, \leq, \perp, u, \sqcup)$ with an observation function $get : \mathbb{S} \rightarrow S$ that materializes the value of a CRDT and satisfies the following:

►(Pierre:) Recall that this is the definition of a state-based CRDT (i.e., a merge semi-lattice).◄

(L1) $get(\perp) = s_0$

(L2) For any $s_1, s_2 \in \mathbb{S}$, $get(s_1 \sqcup s_2) = get(s_1) \oplus get(s_2)$

(L3) get is surjective: for every $s \in S$ there exists some $s' \in \mathbb{S}$ with $get(s') = s$.

(L4) $\sqcup$ is commutative, associative, and idempotent.

Theorem 3.5 (Liftability Characterization). A sequential type T is liftable via $\blacktriangle$ if T is a commutative monoid. ►(Pierre:) Well, this is not really T but instead $(\oplus, S)$.◄

Proof. We prove both directions separately. ($\Rightarrow$) **Necessity:**

Liftability implies commutative monoid

We verify the three axioms of a commutative monoid in turn.

Associativity of $\oplus$. Let $s, t, u \in S$ be arbitrary. By surjectivity (L3), choose $a, b, c \in \mathbb{S}$ with $get(a) = s$, $get(b) = t$, $get(c) = u$.

By (L2) and associativity of $\sqcup$ (L4):

$$get(a \sqcup (b \sqcup c)) = get(a) \oplus get(b \sqcup c) = s \oplus (t \oplus u) \quad (1)$$

$$get((a \sqcup b) \sqcup c) = get(a \sqcup b) \oplus get(c) = (s \oplus t) \oplus u \quad (2)$$

Since $\sqcup$ is associative, $a \sqcup (b \sqcup c) = (a \sqcup b) \sqcup c$, and applying get to both sides:

$$s \oplus (t \oplus u) = (s \oplus t) \oplus u$$

Since s, t, u were arbitrary, $\oplus$ is associative. ►(Pierre:) OK◄

Identity element. Let $t \in S$ be arbitrary and choose $a \in \mathbb{S}$ with $get(a) = t$. We need a state representing s_0 to merge with. By (L1), $get(\perp) = s_0$. We need $\perp$ to act as a fresh contribution with respect to a , which holds at initialization since $\perp$ carries no updates.

By (L2):

$$get(a \sqcup \perp) = get(a) \oplus get(\perp) = t \oplus s_0$$

But $\perp$ is the identity of $\sqcup$ by the CRDT definition, so $a \sqcup \perp = a$, hence:

$$get(a \sqcup \perp) = get(a) = t$$

Therefore $t \oplus s_0 = t$. By a symmetric argument using $\perp \sqcup a = a$:

$$get(\perp \sqcup a) = get(\perp) \oplus get(a) = s_0 \oplus t = t$$

So s_0 is a two-sided identity for $\oplus$. ►(Pierre:) OK◄

Commutativity of $\oplus$. Let $s, t \in S$ be arbitrary. By surjectivity (L3), choose $a, b \in \mathbb{S}$ with $get(a) = s$ and $get(b) = t$.

By (L2):

$$get(a \sqcup b) = get(a) \oplus get(b) = s \oplus t$$

By commutativity of $\sqcup$ (L4):

$$a \sqcup b = b \sqcup a$$

Applying get and (L2) to the right hand side:

$$get(b \sqcup a) = get(b) \oplus get(a) = t \oplus s$$

Since $get(a \sqcup b) = get(b \sqcup a)$, we conclude:

$$s \oplus t = t \oplus s$$

Since s, t were arbitrary, $\oplus$ is commutative.

This completes the proof of necessity. ►(Pierre:) OK◄ ◻

($\Leftarrow$) **Sufficiency: Commutative monoid implies liftability**

Assume that T is a commutative monoid. We construct a valid $\blacktriangle$ -lifting explicitly.

Construction: We define $T^\blacktriangle = (\mathbb{S}, \perp, \Delta')$ as follows:

- **Initial element:** $\perp := \lambda r. s_0$, the constant function returning s_0 .
- **Merge operator:** $(f \sqcup g)(r) := f(r) \oplus g(r)$, applied pointwise.
- **Observation function:** $get(f) := \bigoplus_{r \in R} f(r)$, the fold of $\oplus$ over all replica values.

Construction. Let R be a finite set of replica identifiers

►(Pierre:) One interesting (but not directly related) question is how necessary this assumption is. For instance, what happens is the system is anonymous. This can be useful to bound metadata usage. ►(Ayush:) The replica identifiers here, simplify the get function w.r.t freshness constraints. I will try to find a way to eliminate this.◄

Define:

$$T^\blacktriangle := (R \rightarrow T)$$

that is, the type of functions assigning a value in T to each replica, which we think of as a state vector. Define:

- **Initial element:** $\perp := \lambda r. e$, the constant function returning e .
- **Merge operator:** $(f \sqcup g)(r) := f(r) \oplus g(r)$, applied pointwise.
- **Observation function:** $get(f) := \bigoplus_{r \in R} f(r)$, the fold of $\oplus$ over all replica values.

Each replica r maintains only its own coordinate, so replica r 's update is:

$$\delta_r(f, t)(r') := \begin{cases} f(r') \oplus t & \text{if } r' = r \\ f(r') & \text{otherwise} \end{cases}$$

▷(Pierre:) OK.◀

Verification of CRDT axioms.

Commutativity of $\sqcup$: For any f, g and any r :

$$(f \sqcup g)(r) = f(r) \oplus g(r) = g(r) \oplus f(r) = (g \sqcup f)(r)$$

using commutativity of $\oplus$. Hence $f \sqcup g = g \sqcup f$.

▷(Pierre:) We might need also that e is neutral; no? This is easy: $r, (f \sqcup \perp)(r) = f(r) \oplus \perp(r) = f(r) \oplus e = f(r)$ ◀

▷(Ayush:) The proof of e (now called s_0) being neutral comes from the other direction where we define the CRDT type and prove that it produces a commutative monoid.◀

Associativity of $\sqcup$: Follows immediately from associativity of $\oplus$ applied pointwise.

Idempotency of $\sqcup$: This follows from the disjoint union in the definition of $\oplus$. Since merge is performed pointwise, every operation is only counted once in the binary operator. Thus, remaining idempotent.

Verification of the homomorphism condition (L2). Let $f, g \in T^\Delta$ be fresh with respect to one another, meaning their nonzero coordinates are disjoint: for each r , at most one of $f(r), g(r)$ is non-initial ▷(Pierre:) I see.◀. Then:

$$get(f \sqcup g) = \bigoplus_{r \in R} (f \sqcup g)(r) = \bigoplus_{r \in R} (f(r) \oplus g(r))$$

Since $\oplus$ is commutative and associative, and the supports of f and g are disjoint:

$$= \left(\bigoplus_{r \in R} f(r) \right) \oplus \left(\bigoplus_{r \in R} g(r) \right) = get(f) \oplus get(g)$$

Verification of (L1).

$$get(\perp) = \bigoplus_{r \in R} e = e$$

using the identity law of $\oplus$ iterated over R .

Verification of (L3). For any $t \in T$, fix any replica $r_0 \in R$ and define $f := \lambda r. \text{if } r = r_0 \text{ then } t \text{ else } e$. Then $get(f) = t \oplus e \oplus \dots \oplus e = t$. So get is surjective. ▷(Pierre:) OK◀

This completes the construction and verification. Hence $(T, \oplus, e)$ admits a Δ -lifting. ◻

Conclusion

$(T, \oplus, e)$ admits a Δ -lifting if and only if $(T, \oplus, e)$ is a commutative monoid. ◻

Corollary 3.6 (Non-liftability of string concatenation). *The type $(\Sigma^*, \cdot, \varepsilon)$ admits no Δ -lifting, since string concatenation is not commutative: “AB” · “CD” = “ABCD” $\neq$ “CDAB” = “CD” · “AB”. By the necessity direction of the theorem, no valid get can exist.* ◻

4 Extended Classification of Unliftable Types

4.1 Types with No Binary Operation At All

▷(Pierre:) See my point above about inducing it from the transition function.◀

Consider a type T that carries no natural merge structure whatsoever. The canonical example is:

$$T = \{0, 1\}^* / \sim$$

the set of bitstrings modulo some equivalence that is not a congruence with respect to any commutative operation. More concretely:

Corollary 4.1 (Last-Write-Wins Register). *Let T be any set, with the intended sequential semantics being that the most recent write wins. The “operation” is:*

$$a \oplus b = b \quad (\text{take the rightmost value})$$

This is not commutative since $a \oplus b = b \neq a = b \oplus a$ in general. Therefore no Δ -lifting exists.

However, this is not because the type is unliftable in principle. It is because the sequential specification itself is inherently order-dependent, and no CRDT can implement it without additional metadata. The standard resolution is to enrich T with timestamps, yielding a different sequential type $(T \times \text{Timestamp}, \oplus_{\text{LWW}}, \perp)$ that is liftable. The original T is unliftable but only because it is under-specified. ◻

4.2 Irreparably Unliftable Types

The deepest obstruction arises from types whose semantics are inherently non-monotone. A function $f : T \rightarrow T$ is monotone with respect to a partial order if:

$$a \leq b \Rightarrow f(a) \leq f(b)$$

CRDTs are monotone by construction — state only moves upward in the semilattice order. Therefore:

Theorem 4.2 (Monotonicity Obstruction). *If the sequential specification of $(T, \oplus, e)$ requires any operation that is non-monotone with respect to every partial order on T , then T admits no Δ -lifting.*

The canonical example is:

$$T = (\mathbb{Z}, \min, +\infty)$$

considered not as an output type but as a type supporting decrement without bound. A counter that can decrease arbitrarily has no monotone CRDT representation, because any decrease would require moving downward in the lattice order, contradicting the SEC requirement that state transitions are monotone.

More precisely:

Theorem 4.3 (Non-liftability of unbounded decrement). *Let $T = (\mathbb{Z}, +, 0)$ with the operation set including both increment and decrement as primitive sequential operations rather than as derived from a PN-Counter lifting. Then T admits no Δ -lifting that is monotone and satisfies SEC.*

Proof. Suppose T^Δ exists. Since SEC requires that the state lattice is a join-semilattice, the state space must have a partial

order $\leq$ such that every update is monotone increasing. But the decrement operation $d : T^\Delta \rightarrow T^\Delta$ must satisfy:

$$\text{get}(d(a)) = \text{get}(a) - 1 < \text{get}(a)$$

So d strictly decreases the observable value. For d to be monotone, we need $d(a) \geq a$ in the lattice order, yet get must be order-preserving, so $\text{get}(d(a)) \geq \text{get}(a)$, contradicting $\text{get}(d(a)) = \text{get}(a) - 1$. Therefore no such lifting exists. $\square$

Remark. The PN-Counter resolves this not by lifting $(\mathbb{Z}, +, 0)$ directly, but by changing the type to a pair of monotone counters $(\mathbb{N} \times \mathbb{N}, \oplus_{\text{PN}}, (0, 0))$ whose get computes the difference. The integer type itself remains unliftable.

Type class	Liftable?	Reason
Commutative monoids ($\mathbb{N}, +, 0$), ($\mathbb{B}, \vee, \perp$)	✓ Yes	Constructive lifting exists (Theorem 1)
Non-commutative monoids ($\Sigma^*, \cdot, \epsilon$)	× No	Commutativity of $\sqcup$ forces commutativity of $\oplus$
Non-associative types ($\mathbb{Z}, -, 0$)	× No	Associativity of $\sqcup$ forces associativity of $\oplus$
Non-monotone types ($\mathbb{Z}, +, 0$) with decrement	× No	SEC monotonicity is violated
Under-specified types (LWW register)	× No directly ✓ with enrichment	Liftable only after adding metadata to the type

Table 1. classification of types with respect to liftability